

FUNDAMENTOS Y APLICACIONES DE BLOCKCHAINS

Homework 1

Depto. de Computación, UBA, 2do. Cuatrimestre 2025

4/9/25

Student:

Due: 18/9/25, 17:00 hs

Instructions

- Upload your solution to Campus; make sure it's only one file, and clearly write your name on the first page. Name the file '<your last name>_HW1.pdf.'

If you are proficient with $\text{\LaTeX}$, you may also typeset your submission and submit in PDF format. To do so, uncomment the "`%\begin{solution}`" and "`%\end{solution}`" lines and write your solution between those two command lines.

- Your solutions will be graded on *correctness* and *clarity*. You should only submit work that you believe to be correct.
- You may collaborate with others on this problem set. However, you must **write up your own solutions** and **list your collaborators and any external sources (including ChatGPT and similar generative AI chatbots)** for each problem. Be ready to explain your solutions orally to a member of the course staff if asked.

This homework contains 7 questions, for a total of 80 points.

1. This question is about Merkle Trees (MTs).

(a) (3 points) Describe how (cryptographic) hash functions are used in a MT.

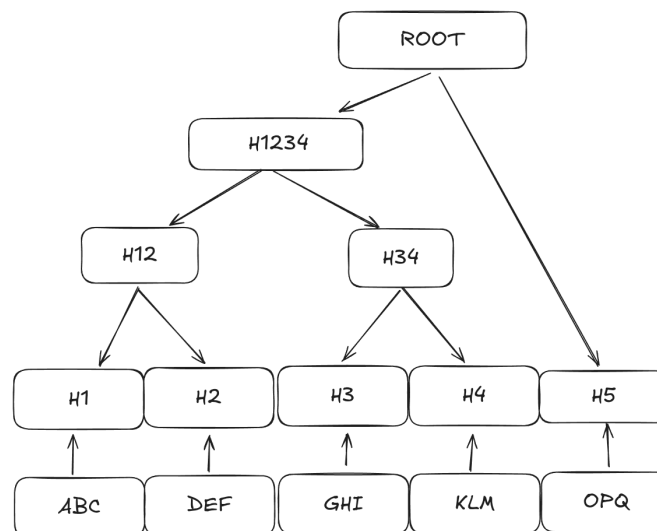
Solution: Un árbol de Merkle usa funciones hash para darle valores únicos a cada chunk. Dado un bloque grande de datos, lo partimos en chunks. Cada uno de esos chunks tiene un hash asociado. Luego, ese arreglo de hashes de cada chunk van a ser las hojas del árbol. Las hojas número 1 y 2 van a tener un padre, que va a ser el hash de la combinación de ellos mismos (es decir, de sus hashes). Lo mismo para las hojas 3 y 4, etc.

Ahora, hacemos lo mismo para los padres de estas hojas, formando así los abuelos de las hojas originales. Cada uno de estos abuelos será padre de dos padres, combinando sus hashes. Repetimos estos pasos hasta llegar a la raíz del árbol.

De este modo, conseguimos un identificador del árbol en muy poco espacio. Esto nos va a servir para las pruebas de inclusión, que nos permiten validar si el hash de un chunk que, a priori, no sabemos si es el correcto. Quien quiera verificarlo tiene que reconstruir el árbol de hashes, asumiendo que son dados los hashes necesarios de cada bloque. Si el resultado final (la raíz) es la misma que nos habíamos guardado, hemos verificado que el chunk que nos dieron era el mismo que teníamos antes.

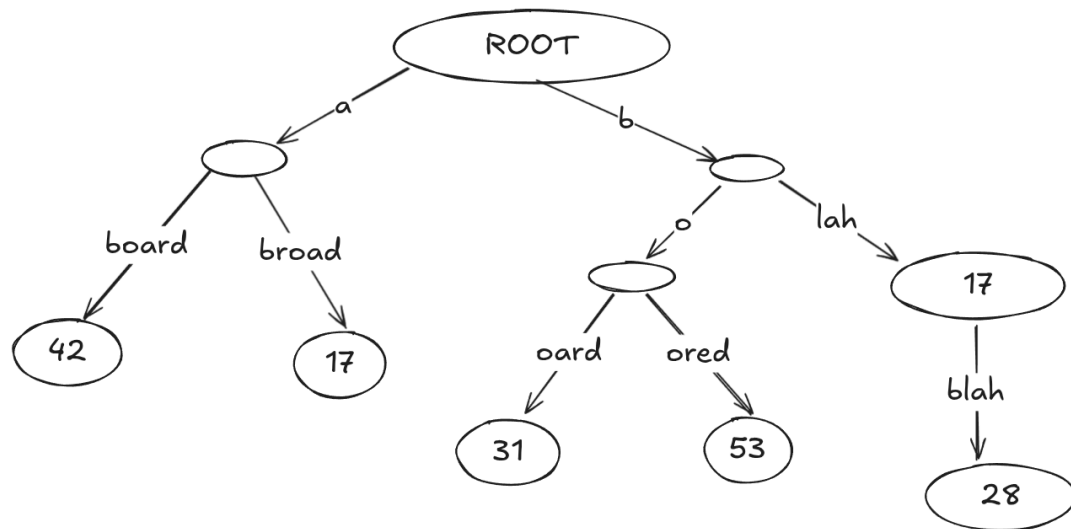
(b) (3 points) Describe how a (complete, binary) MT is constructed for the following five chunks of data: ABC, DEF, GHI, KLM, OPQ.

Solution: En este caso, al ser una cantidad de chunks impares, tenemos que decidir qué vamos a hacer. En mi solución, yo opté por **no duplicar los hashes del bloque impar**.



- (c) (4 points) Describe how a Patricia Trie is constructed for the following key/value store: {blah: 17, blahblah: 28, bored: 53, board: 39, aboard: 42, abroad: 17}.

Solution:



2. Cryptographic hash functions:

- (a) (5 points) Derive the formula for the *birthday paradox* we saw in class (show your work, explaining every step) and calculate the number of elements needed to find a collision with at least 50%.

Solution: Dado que tenemos n posibles fechas y k personas, la probabilidad de que dos personas tengan la misma fecha de cumpleaños es la siguiente. Tomamos B como el evento de que todos tengan fechas de cumpleaños diferentes. Luego, tomar $1 - P(B)$ es la probabilidad de que haya una colisión.

Notamos que la primera persona puede elegir cualquiera (n posibilidades).

Luego, la segunda persona puede elegir $n - 1$ fechas.

Y así sucesivamente.

$$P(B) = \frac{n}{n} \cdot \frac{n-1}{n} \cdot \frac{n-2}{n} \cdot \dots \cdot \frac{n-k+1}{n}$$

Luego, lo que nos queda en el numerador es una permutación sin repeticiones.

$$P(B) = \frac{n!}{(n-k)!} \cdot \frac{1}{n^k}$$

Ahora reescribimos la expresión anterior como un producto. Cada factor $\frac{n-i+1}{n}$ se puede reescribir como $1 - \frac{i-1}{n}$, donde i va de 1 a k .

$$P(B) = \prod_{i=0}^{k-1} \left(1 - \frac{i}{n}\right)$$

Luego, observemos que cada uno de los elementos de la productoria se ve así $\frac{1-i}{n}$. A estos los podemos acotar superiormente por $e^{-\frac{i}{n}}$.

$$P(B) \leq \prod_{i=0}^{k-1} e^{-\frac{i}{n}}$$

Luego, reescribimos la productoria como una exponencial.

$$P(B) \leq \prod_{i=0}^{k-1} \exp\left(-\frac{i}{n}\right) = \exp\left(\frac{-1}{n} \cdot \sum_{i=1}^{k-1} i\right)$$

Luego, usamos la propiedad de la suma de los primeros $k - 1$ enteros.

$$P(B) \leq \exp\left(\frac{-1}{n} \cdot \frac{(k-1)k}{2}\right)$$
$$P(B) \leq \exp\left(\frac{-k(k-1)}{2n}\right)$$

Luego, despejamos k a partir de $P(B) \leq \exp\left(-\frac{k(k-1)}{2n}\right)$.

$$\ln\left(\frac{1}{P(B)}\right) \geq \frac{k(k-1)}{2n}$$
$$k \geq \sqrt{2n \ln\left(\frac{1}{P(B)}\right)}$$

- (b) (5 points) Apply the above result to find out how many Bitcoin users are needed to initialize their wallet's seed, which is based on a random selection of 12 random words from the list in <https://github.com/bitcoin/bips/blob/master/bip-0039/english.txt>, to have the event that, with probability at least 50%, at least two users end up with exactly the same seed.

Solution: La idea es usar la fórmula anterior para despejar k (la cantidad de usuarios). Vemos que esa lista tiene 2048 palabras. Si cada usuario elige 12 palabras de manera aleatoria, y pueden repetirse, tenemos que $n = 2048^{12} = 2^{132}$. Para colisión con probabilidad al menos 50%, tomamos $P(B) = 0.5$.

$$k \geq \sqrt{2n \ln\left(\frac{1}{P(B)}\right)} = \sqrt{2 \cdot 2^{132} \ln 2}$$
$$\approx 8.7 \times 10^{19}$$

3. (10 points) Prof. Gray has designed a cryptographic hash function $H_G : \{0,1\}^* \rightarrow \{0,1\}^n$. One of his brilliant ideas is to make $H_G(x) = x$ if x is an n -bit string (assume the behavior of H_G is much more complicated on inputs of other lengths). That way, we know with certainty that there are no collisions among n -bit strings. Has Prof. Gray made a good design decision? Justify your answer, in particular listing the properties that may or may not be satisfied by the design.

Solution: La solución no es buena. En particular, algunas de las propiedades que **no** cumple son las siguientes:

- **Resistencia a preimagen:** Dado $y = H_G(x)$ no es factible encontrar algún x' tal que $H_G(x') = y$.

En este caso (tomando que x es un string de n bits), si alguien conoce solamente el hash de un input, no es para nada difícil encontrar el input que produce ese hash. Se reduce a probar el mismo hash como input y se verifica la propiedad.

Siguiendo la definición que dimos, sería tomar $x' = x = y$. Así se verificaría que $H_G(x') = y$.

- **Resistencia a segunda preimagen:** Dado x , no es factible encontrar $x' \neq x$ tal que $H_G(x) = H_G(x')$.

Esto tampoco lo cumple. En el caso de que x **no** sea un string de n bits, es factible encontrarlo. Tomemos $x' = H_G(x)$. Por definición de H_G , su codominio es de n bits. Por lo tanto, $x' \neq x$ (pues x no es de n bits) y vemos que $H_G(x') = H_G(H_G(x)) = H_G(x)$.

En el caso de que x sea un string de n bits, es más difícil. Ahí sí tenemos que buscar sobre todo el espacio de los strings que no sean de n bits para encontrar otro.

- **Pequeños cambios en el input producen grandes cambios en el output:** En este caso, si x es un string de n bits, y x' es un string de n bits que difiere en un bit, $H_G(x) \neq H_G(x')$ (es decir, no hay colisión), pero difieren solamente en un bit. Esto no cumple, ya que se espera que sea un cambio mayor.

4. (10 points) As we saw in class, the main security notion for digital signatures is called *existential unforgeability*, which prevents an attacker from producing a signature on a message that has not been produced by the legitimate signer. I.e., the attacker should not be able to produce the pair (m, σ) , where σ was not produced by the legitimate signer.

Show an attack on the plain RSA signature scheme we saw in class in which an attacker forges a signature on an arbitrary message m by asking the signer to sign two other different messages (not necessarily unrelated to m).

Solution: Dado un mensaje arbitrario m , queremos mostrar como un atacante podría falsificar una firma de este (llamémosla σ), pidiéndole que firme dos mensajes diferentes. Sea (N, e) la clave pública y d la clave privada.

Nuestro objetivo es construir σ tal que $\sigma = m^d \pmod N$.

Tomemos m_1 y m_2 dos mensajes diferentes que nos pide la consigna. Todavía no sabemos qué propiedad van a cumplir, pero sabemos que podemos obtener su firma. Luego, le pedimos al firmante que firme estos mensajes. Llamamos a las firmas de estos mensajes σ_1 y σ_2 . Cumplen que $\sigma_1 = m_1^d \pmod N$ y $\sigma_2 = m_2^d \pmod N$.

Volvamos a nuestro objetivo inicial. Queremos que $\sigma = m^d \pmod N$. Qué pasa si tomamos $\sigma = \sigma_1 \cdot \sigma_2$?

$$\begin{aligned}\sigma &= \sigma_1 \cdot \sigma_2 \\ &= (m_1^d \pmod N) \cdot (m_2^d \pmod N) \\ &= (m_1 \cdot m_2)^d \pmod N\end{aligned}$$

Luego, para llegar a nuestro objetivo de $\sigma = m^d \pmod N$, nos basta con que $m = m_1 \cdot m_2 \pmod N$

Luego, logramos construir una firma para m usando dos firmas de dos mensajes diferentes entre sí (y diferentes de m).

5. (10 points) A Bitcoin miner creates a block B which contains address α , on which it wants to receive its rewards. An attacker changes the contents of B , such that instead of α it defines a new address α' , which is controlled by the attacker. Will the attacker receive the rewards that the miner tries to claim? Why or why not? Give a detailed explanation of your answer.

Solution: El problema plantea lo siguiente: un minero legítimo crea un bloque B donde se guarda la dirección α , donde el minero quiere recibir sus recompensas.

Luego, un atacante cambia el contenido de B , de manera que en lugar de α , define una nueva dirección α' , controlada por él. Veamos por qué el atacante no recibirá las recompensas que el minero intenta reclamar.

El bloque en sí mismo tiene un hash asociado, que se computa usando el contenido del bloque. Entonces, si el bloque cambia, el hash va a cambiar también. Un bloque es válido si su hash es menor que el *target* actual.

El resto de nodos (asumimos legítimos) de la red de Bitcoin, al recibir ese bloque con la dirección modificada, van a computar el hash del bloque que recibieron y van a ver que no cumple con la dificultad requerida (es decir, que sea menor que el *target* actual). Por lo tanto, el bloque no es válido.

Si el atacante quisiera hacer que este bloque sea válido, tendría que modificar el *nonce* del bloque, ya que el hash actual (el que incluye a α' como dirección) no va a ser válido puesto que no va a cumplir la dificultad requerida (no va a ser menor que el *target* actual).

El rol que cumple el *nonce* es de un contador que se incrementa en 1 en cada intento de hash. En este intento se prueba si el hash con el *nonce* actual es menor que el *target*. Si es menor, el bloque es válido y se puede añadir a la cadena. Si no, se incrementa el *nonce* y se vuelve a intentar.

Luego, para obtener un nuevo *nonce* válido, el atacante tendría que hacer fuerza bruta hasta encontrar un *nonce* que, cuando se le hace el hash a todo el bloque (proceso que incluye el *nonce* como input de esa función hash), produzca un hash menor que el *target* actual.

Para cuando el atacante haya encontrado un *nonce* válido, la red legítima va a haber avanzado con el siguiente bloque y no va a aceptar el bloque modificado por el atacante.

De este modo concluimos que el atacante no recibirá las recompensas que el minero intenta reclamar.

6. (10 points) Using the course's Sepolia Testnet chain, send 0.05 ETH to the address of a fellow student. Describe how you conducted the payment, including the transaction's id and addresses which you used. (Refer to the Solidity tutorial material "How to Connect to a Public Testnet" document shared on Campus, as well as the Solidity tutorial material and pointers.)

Solution: Mi dirección es 0xEE267E171c91F70C492bE8061D31aF5C5754F4c.

Esta es la dirección del contrato que usé para enviarle 0.05 ETH a mi compañero.

El código fuente del contrato es el siguiente:

```
// SPDX-License-Identifier: GPL-3.0

pragma solidity >=0.7.0 <0.9.0;

contract SimpleTransactionSender {
    function send(address to_address) public payable {
        payable(to_address).transfer(msg.value);
    }
}
```

Como se puede observar, es muy simple y su única función es enviar ETH a una dirección. Luego, usé el método `send` del contrato, con su dirección como parámetro. Las interacciones (al igual que el *deploy*) los hice usando Remix.

Finalmente, esta es la transacción en Sepolia Etherscan.

7. The following smart contract has been deployed on the Sepolia Testnet chain:

```
// SPDX-License-Identifier: GPL-3.0

pragma solidity >=0.7.0 <0.9.0;

contract Bank {
    mapping(address => uint256) balance;
    address[] public customers;

    event Deposit(address customer, string message);
    event Withdrawal(address customer);

    function deposit(string memory message) public payable {
        require(msg.value > 10);
        balance[msg.sender] += msg.value - 10;

        customers.push(msg.sender);

        emit Deposit(msg.sender, message);
    }

    function withdraw() public {
        uint256 b = balance[msg.sender];
        balance[msg.sender] = 0;
        payable(msg.sender).transfer(b);

        emit Withdrawal(msg.sender);
    }

    function getBalance() public view returns (uint256) {
        return balance[msg.sender];
    }

    function empty() public {
        require(msg.sender == customers[0]);

        payable(msg.sender).transfer(address(this).balance);
    }
}
```

Its deployed address is: 0x4310a5f4Bb3b4Ff09f0694218fA50D2767a00b86.

You can compile it and interact with it using Remix. You should successfully create a transaction that interacts with the contract, either depositing or withdrawing from it some coins.

- (a) (5 points) Describe the contract's functionality (that is, the purpose of each variable, function, and event).

Solution: En general, el contrato es una especie de "Banco" que permite a los usuarios depositar y retirar fondos. Veamos parte por parte que hace.

- `mapping`: es un diccionario que guarda, para cada dirección, el saldo de la cuenta.
- `customers`: es un arreglo que guarda todas las direcciones que han depositado o retirado fondos.
- `Deposit`: es un evento que se emite cuando un usuario deposita fondos. Además de la dirección del cliente, tiene un mensaje.
- `Withdrawal`: es un evento que se emite cuando un usuario retira fondos.
- `deposit`: es una función que permite a un usuario depositar fondos. Tiene un mensaje como parámetro, que es lo que va a recibir el evento `Deposit`. Luego, se agregan los fondos al balance de la persona que envió el dinero, restando 10 wei (por eso se requiere que se envíen mas de 10 wei). Algo llamativo es que se agrega la dirección del sender al arreglo de clientes, sin chequear si ya estaba ahí.
- `withdraw`: es una función que permite a un usuario retirar fondos. Su objetivo es vaciar el balance del que llama. Luego, se llama a `payable(msg.sender).transfer(b)`. Esto hace que podamos llamar a `transfer` para poder enviar el balance del usuario `b` (que es la cantidad que tenía el usuario antes de llamar a la función). Finalmente, se emite el evento `Withdrawal` con la dirección del cliente.
- `getBalance`: es una función que permite a un usuario ver su balance.
- `empty`: es una función que permite al usuario 0 vaciar todo el balance del contrato. Vemos que se requiere que el usuario que llama sea el primer cliente (el que está en la posición 0 del arreglo `customers`). Luego, se convierte la dirección del que llama a una dirección payable, para luego llamar a `transfer` para enviar el balance del contrato. Este balance total del contrato se obtiene con `address(this).balance`.

(b) (15 points) Provide the id of the transaction you performed and the address you used.

Solution: El id de la transacción (en la que envié 100 wei) es:

0x1044bd3202eef32b72976c02c035b0992b28c6f65f1d743a1a2070902de4c493.

Mi dirección es 0xEEE267E171c91F70C492bE8061D31aF5C5754F4c.